

Chapter 4

Addressing Modes

Learning Outcomes

After completion of this chapter students will know

- ❖ Different methods of manipulating data among registers, memory and I/O devices in an 8086 system
- ❖ Alternate methods of accessing data in memory
- ❖ Special modes of accessing data in stacks
- ❖ Special modes of accessing and manipulating strings

4.1 Introduction

8086 is a complex instruction set (CISC) processor. It has powerful instructions that can carry out complex tasks in a single instruction. 8086 has an elaborated instruction set that supports many modes of work to ease the task of application programming.

4.2 8086 Addressing Modes

8086 has 117 different instructions with about 300 opcodes. The instructions are categorized in 12 basic addressing modes classified into 5 groups.

1. Accessing immediate and register data
 - a. Register addressing mode
 - b. Immediate addressing mode
2. Accessing data in memory
 - a. Direct addressing mode
 - b. Register indirect addressing mode
 - c. Based addressing mode
 - d. Indexed addressing mode
 - e. Based indexed addressing mode
 - f. String addressing mode
3. Accessing I/O ports
 - a. Direct accessing
 - b. Indirect accessing
4. Relative addressing
5. Implied addressing

4.3 Accessing immediate and register data

4.3.1 Register Addressing Mode

In this mode both operands are registers and are useful for data transfer between registers using MOV instruction. Both registers should be of same size.

Example 4:1

MOV DX, CX ; Move the content of CX register into DX register
MOV AH, DL ; Move contents of DL register into AH

4.3.2 Immediate Addressing Mode

In this mode an 8-bit or 16-bit data can be specified as part of the instruction.

Example 4:2

MOV DL, 80H ; Move 80H into DL register
MOV AX, 8045H ; Move 8045H into AX, 80H → AH and 45H → AL

Example 4:3

VALUE EQU 80H
MOV AX, VALUE ; Move 80H into AX, 80H ← VALUE
Thus the data may have an immediate value or may be a constant defined by the EQU directive.

4.4 Accessing Data in Memory

4.4.1 Direct Register Addressing Mode

In this mode data is transferred between register and memory in the data segment. One operand should be a register and the other one as the effective address (EA, offset in the data segment) in the data segment.

Example 4:4

MOV CX, OFFSET1 ; Move 16bit data from OFFSET1 location in the data segment
Physical address = $[DS]*16 + \text{OFFSET1}$ (in the data memory)
MOV START, AX ; Move 16-bit data from register AX to memory at EA = START
Physical location of START = $[DS]*16 + \text{START}$
 $[DS]*16 + \text{START} \leftarrow \text{AL}$
 $[DS]*16 + \text{START} + 1 \leftarrow \text{AH}$

4.4.2 Register Indirect Addressing Mode

In this mode EA should be either a pointer register (BX or BP) or an index register (SI or DI).

Example 4:5

MOV [DI], BX ; Move BX content into data segment in the EA addressed by DI
 $[DS]*16 + [DI] \leftarrow \text{BX}$

4.4.3 Based Addressing Mode

In this mode one operand is base register (BX) or base pointer (BP). For BX the data location is the data segment, however, for base pointer the data location is in the stack segment.

Example 4:6

MOV AL, DISP [BX] ; Move data from EA of $\text{DISP} + [BX]$ to AL
 $\text{AL} \leftarrow [DS]*16 + [BX] + \text{DISP}$
MOV [BP-6], AX ; Move data from AX register into Stack Segment at EA = [BP-6]
 $[SS]*16 + [BP-6] \leftarrow \text{AX}$

4.4.4 Indexed Addressing Mode

In this mode one operand is the index registers SI or DI with a displacement.

Example 4:7

```
MOV AL, DISP [SI] ; Move data from EA of [SI] + DISP in the DS to AL
                  ;  $AL \leftarrow [DS]*16 + [SI] + DISP$ 
MOV DISP [DI], AL ; Move data from AL to EA of [SI] + DISP in the DS
                  ;  $AL \rightarrow [DS]*16 + [DI] + DISP$ 
```

4.4.5 Based Indexed Addressing Mode

In this mode, the EA is computed adding a base register (BX or BP), index register (SI or DI), and a displacement.

Example 4:8

```
MOV AL, DISP [SI] [BX] ; Move data from EA of [BX]+[SI] + DISP in the DS to AL
                       ;  $AL \leftarrow [DS]*16 + [BX] + [SI] + DISP$ 
MOV DISP [DI] [BX], AL ; Move data from AL to EA of [BX] + [SI] + DISP in the DS
                       ;  $AL \rightarrow [DS]*16 + [BX] + [DI] + DISP$ 
MOV AL, DISP [SI] [BP] ; Move data from EA of [BP] + [SI] + DISP in the SS to AL
                       ;  $AL \leftarrow [SS]*16 + [BP] + [SI] + DISP$ 
MOV DISP [DI] [BP], AL ; Move data from AL to EA of [BP] + [SI] + DISP in the SS
                       ;  $AL \rightarrow [SS]*16 + [BP] + [DI] + DISP$ 
```

4.4.6 String Addressing Mode

This mode uses index registers, SI and DI. The source data is in DS and destination data in ES. For string instructions, SI or DI are automatically incremented if DF = 0 (DF is cleared by CLD and set by STD) and decremented if DF = 1.

Example 4:9

```
CLD ; Clear direction flag, DF = 0
MOVS BYTE ; Move string data from EA of [SI] in DS to EA of
          ; [DI] in ES
          ;  $[ES]*16 + [DI] \leftarrow [DS]*16 + [SI]$ 
          ;  $[SI] = [SI] + 1$  and  $[DI] = [DI] + 1$ 
MOVS WORD ;  $[ES]*16 + [DI]$ 
          ;  $\leftarrow [DS]*16 + [SI]$ 
          ;  $[SI] = [SI] + 2$  and  $[DI] = [DI] + 2$ 
```

4.5 Addressing Modes for Accessing I/O Ports

4.5.1 Direct Addressing I/O Ports

In this mode, in the IN/OUT instructions, the port address is an 8-bit direct data. One of the operand is a register, either AL (for 8-bit data) or AX (for 16-bit data). In this mode only 256 ports can be accessed.

Example 4:10

```
IN AL, 80H ; Read 8 bit data from port 80H and move it to [AL]
IN AX, 80H ; Read 16bit data from port 80H and move it to [AX]
OUT 81H, AL ; Write 8bit data from [AL] to port with address 81H
```

4.5.2 Indirect Addressing I/O Ports

In this mode port address is assigned in DX register. Here also only AL or AX register can be used for data. In these mode upto 64K 8-bit ports or 32K 16-bit ports can be accessed.

Example 4:11

```
IN AL, DX      ; Read 8-bit data from port addressed by [DX] and move it to [AL]
                ; [AL] ← [[DX]]
IN AX, DX      ; Read 16-bit data from port addressed by [DX]
                ; and move it to [AX], [AX] ← [[DX]]
OUT DX, AL     ; Write 8-bit data from [AL] to port with addressed
                ; by [DX], [[DX]] ← [AL]
```

4.6 Relative Addressing Mode

In this mode, the operand is an 8-bit signed displacement relative to the IP.

Example 4:12

```
JNZ LOOP1     ; If zero flag is not set then [IP] = [IP] + LOOP1
                [IP] = [IP] + 1
                ; otherwise execute the next instruction
```

4.7 Implied Addressing Mode

In this mode there are no operands.

Example 4:13

```
CLC           ; Clear carry flag
CLI           ; Clear interrupt enable flag
CMC           ; Complement carry flag (1's complement)
```

Review Questions

1. What does the term addressing mode mean?
2. How many addressing modes do 8086 supports? Name the different addressing modes supported by 8086.
3. Explain the addressing modes of 8086 relating to addressing of data in data segment.
4. How 8086 does addresses data in stack segment?
5. Discuss the different indexed addressing modes in 8086 with one example each.
6. What is based indexed addressing mode? Give two examples and explain their functionalities.
7. Explain in detail the string addressing mode in 8086.
8. How does 8086 accesses I/O ports having 8-bit and 16-bit addresses.